Задача 1:

Вие сте разработчик на back-end компоненти за система за управление на резервации на хотелски стаи. Всеки ден се добавят нови стаи за резервация в система, а потребителите могат да правят резервации за различни типове стаи в зависимост от техните нужди. Всеки ден след като приключат сесиите за резервации, информацията за стаите и техните наличности се обновява и системата подготвя новите стаи за продажба на следващия ден.

Задача 1.1:
Създайте клас **Room**, който да представлява общ тип стая в хотел. Класът трябва да съдържа следните атрибути:

- roomNumber (номер на стаята)
- roomType (тип стая: например "единична", "двойна", "апартамент")
- pricePerNight (цена за нощувка)
- availableNights (броят на наличните нощувки за тази стая)

Класът трябва да има:

- **Конструктор** с параметри за инициализация на атрибутите.
- Метод boolean reserveRoom(int numNights) throws NoAvailableNightsException, който резервира даден брой нощувки за стаята:
    - Намалява броя на наличните нощувки с подадената стойност.
    - Връща true, ако резервирането е успешно.
    - Хвърля изключение **NoAvailableNightsException**, ако няма достатъчно налични нощувки.

Създайте два наследника на класа **Room**:

1. **SingleRoom** – клас, който представлява единична стая.
2. **SuiteRoom** – клас, който представлява апартамент (по-голяма стая с допълнителни удобства).

Всеки от класовете трябва да съдържа специфични за типа стая допълнителни характеристики, като например:

- **SingleRoom** може да съдържа допълнителен атрибут за категория на леглото (например "единично", "голямо единично легло").
- **SuiteRoom** може да съдържа допълнителен атрибут за брой на стаите в апартамента (например спалня, хол, кухня).

Класът Room трябва да имплементира Serializable. Обектите от класовете наследници също ще могат да се сериализират, защото наследяват от Room.

Задача 1.2:
Създайте интерфейс **Reservable**, който да съдържа метод:

- void updateAvailabilityFromFile() throws IOException: Този метод ще чете наличните стаи и техните резервации от файл и ще ги обновява в системата. Възможно е да възникне грешка при четенето на файла, която трябва да се обработи.

Създайте клас **RoomLoader**, който имплементира интерфейса **Reservable**. Класът трябва да чете информация от файл с име rooms.txt, който съдържа информация за стаите и техните наличности. Форматът на файла трябва да е следният:

1*101*Single*50*10
2*102*Suite*150*5
1*103*Single*60*8
2*104*Suite*200*3

- Първата цифра на всеки ред определя дали стаята е **единична** (1) или **апартамент** (2).
- Следващите полета съдържат номера на стаята, типа на стаята, цената за нощувка и броя налични нощувки.

Ако файлът съдържа невалиден ред или не може да бъде прочетен, методът трябва да хвърли и обработи изключение **InvalidRoomDataException** (за невалиден ред или неправилни данни във файла). Ако възникне грешка при четенето на файла (например файлът не съществува или има проблеми с формата), методът трябва да хвърли **IOException**.

Задача 1.3: Сериализиране на обектите

След като стаите бъдат прочетени от rooms.txt и създадени като обекти:

1. Запишете всички стаи във файл rooms.ser.
2. Използвайте ObjectOutputStream и метода writeObject().
3. Запишете масив от тип Room[], съдържащ както SingleRoom, така и SuiteRoom.

Задача 1.4: Десериализиране на обектите

Прочетете масива обратно от rooms.ser:

1. Използвайте ObjectInputStream и метода readObject().
2. Преобразувайте прочетения обект към Room[].
3. Изведете стаите и техните данни на екрана.

Задача 1.5:
Създайте клас **RoomBookingSystem**, който съдържа два публични масива:

- **singleRooms** (масив от единични стаи)
- **suiteRooms** (масив от апартаменти)

Имплементирайте статичен метод void processRoomReservations(), който:

- Използва **RoomLoader** за зареждане на стаи от файла.
- Разделя заредените стаи в два масива – единият с единични стаи, а другият с апартаменти.
- За да разделите стаите, използвайте `instanceof`.

Методът трябва да обнови наличностите на стаите и да изведе съобщение за всяка успешна или неуспешна резервация.